Experiment No. 9
To implement policy gradient methods, such as REINFORCE or Proximal Policy Optimization (PPO), to train an agent in a continuous control environment.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement policy gradient methods, such as REINFORCE and Proximal Policy Optimization (PPO), to train an agent in a continuous control environment and analyze their performance in learning optimal policies.

Objective:

- To understand the concept of **policy-based reinforcement learning methods**.
- To implement the **REINFORCE algorithm** for direct policy optimization.
- To implement **Proximal Policy Optimization (PPO)** for stable and efficient learning.
- To train an agent in a **continuous action space environment**.

Theory:

1. Policy-Based Reinforcement Learning

Unlike value-based methods, policy-based methods directly learn a **policy function**:

$$\pi(a \mid s; \theta)$$

Where:

- $\pi(a \mid s; \theta)$: Probability of taking action $a$in state s
- θ: Parameters of the policy

Goal: Maximize expected cumulative reward:

$$J(\theta) = \mathbb{E}[R]$$

2. Policy Gradient Theorem

The policy is updated using gradient ascent:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi(a \mid s; \theta) \cdot R]$$

This means:

- Increase probability of actions that yield higher rewards
- Decrease probability of poor actions

3. REINFORCE Algorithm

Concept

REINFORCE is a **Monte Carlo policy gradient method**:

- Uses complete episodes to update policy
- No value function required

Steps

1. Generate an episode using current policy
2. Compute total return R
3. Update policy parameters using gradient

Advantages

- Simple to implement
- Works well for small problems

Limitations

- High variance in updates
- Slow convergence
- Unstable learning

4. Proximal Policy Optimization (PPO)

Concept

PPO improves stability by limiting large policy updates.

Key Idea

- Restrict how much the policy can change in one update
- Uses a **clipped objective function**

$$L^{CLIP}(\theta) = \mathbb{E}[\min(r(\theta)A, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)A)]$$

Where:

- $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$
- A: Advantage function
- ϵ: Clipping parameter

Implementation:

```
import gym

import numpy as np

import tensorflow as tf

from tensorflow.keras import layers



env = gym.make("CartPole-v1")



state_size = env.observation_space.shape[0]
```

```
action_size = env.action_space.n

gamma = 0.99

learning_rate = 0.001


# Policy Network

model = tf.keras.Sequential([

    layers.Dense(24, activation='relu', input_shape=(state_size,)),

    layers.Dense(24, activation='relu'),

    layers.Dense(action_size, activation='softmax')

])


optimizer = tf.keras.optimizers.Adam(learning_rate)


# Training

episodes = 200


for ep in range(episodes):

    state = env.reset()[0]

    states, actions, rewards = [], [], []


    done = False

    while not done:
```

```
        state_input = np.reshape(state, [1, state_size])

        probs = model(state_input).numpy()[0]

        action = np.random.choice(action_size, p=probs)



        next_state, reward, done, _, _ = env.step(action)



        states.append(state)

        actions.append(action)

        rewards.append(reward)



        state = next_state


    # Compute returns

    returns = []

    G = 0

    for r in reversed(rewards):

        G = r + gamma * G

        returns.insert(0, G)


    # Normalize returns

    returns = np.array(returns)

    returns = (returns - np.mean(returns)) / (np.std(returns) + 1e-8)
```

```
    # Update policy
    with tf.GradientTape() as tape:
        loss = 0
        for i in range(len(states)):
            state_input = np.reshape(states[i], [1, state_size])
            probs = model(state_input)
            action_prob = probs[0, actions[i]]
            loss += -tf.math.log(action_prob) * returns[i]

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))

    print(f"Episode {ep+1}, Total Reward: {sum(rewards)}")
```

```
import gym
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
```

```
state_size = env.observation_space.shape[0]

action_size = env.action_space.n


gamma = 0.99

epsilon = 0.2

learning_rate = 0.001


# Actor-Critic Model

state_input = layers.Input(shape=(state_size,))

dense = layers.Dense(64, activation='relu')(state_input)

dense = layers.Dense(64, activation='relu')(dense)


action_probs = layers.Dense(action_size, activation='softmax')(dense)

state_value = layers.Dense(1)(dense)


model = tf.keras.Model(inputs=state_input, outputs=[action_probs, state_value])

optimizer = tf.keras.optimizers.Adam(learning_rate)


# Training Loop

episodes = 100
```

```
for ep in range(episodes):
    state = env.reset()[0]
    states, actions, rewards = [], [], []

    done = False
    while not done:
        state_input = np.reshape(state, [1, state_size])
        probs, _ = model(state_input)
        probs = probs.numpy()[0]

        action = np.random.choice(action_size, p=probs)

        next_state, reward, done, _, _ = env.step(action)

        states.append(state)
        actions.append(action)
        rewards.append(reward)

        state = next_state

    # Compute returns
    returns = []
```

```
G = 0
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)

returns = np.array(returns)

# Training
with tf.GradientTape() as tape:
    total_loss = 0

    for i in range(len(states)):
        s = np.reshape(states[i], [1, state_size])
        probs, value = model(s)
        action_prob = probs[0, actions[i]]

        advantage = returns[i] - value[0]

        ratio = action_prob / (action_prob + 1e-10)
        clipped = tf.clip_by_value(ratio, 1-epsilon, 1+epsilon)

        loss = -tf.minimum(ratio * advantage, clipped * advantage)
```

```
        total_loss += loss

    grads = tape.gradient(total_loss, model.trainable_variables)

    optimizer.apply_gradients(zip(grads, model.trainable_variables))


    print(f"PPO Episode {ep+1}, Reward: {sum(rewards)}")
```

Conclusion:

Policy gradient methods directly optimize the policy and are suitable for continuous action spaces.

- **REINFORCE** is simple but suffers from high variance and instability.
- **PPO** improves performance by stabilizing updates using clipping.

Thus, PPO is widely used in modern reinforcement learning due to its efficiency and reliability.